

Contents

Components:	2
React.js Frontend:	2
API Gateway:	2
Node.js Backend:	2
MongoDB:	2
Oracle/PostgreSQL:	2
Workflow:	2
Benefits of this architecture:	3
Improved security:	3
Scalability:	3
Flexibility:	3
Reduced complexity:	3
Additional considerations:	3
Diagram.....	4

Amr System overview

Components:

React.js Frontend:

- Handles user interface, interacts with API Gateway for authorized data access and displays graphical reports.

API Gateway:

- (e.g., Kong, Tyk) Authenticates users using various protocols (OAuth, etc.), authorizes requests based on defined policies, and routes them to appropriate backend services.

Node.js Backend:

- Handles Modbus operations (meter readings), interacts with MongoDB (Modbus readings) and Oracle/PostgreSQL (DLMS readings), fulfills authorized requests from the API Gateway.

MongoDB:

- Stores meter readings received via Modbus protocol.

Oracle/PostgreSQL:

- Stores meter readings received via DLMS protocol.

Workflow:

- User interacts with the React.js frontend, triggering a data request.
- Frontend sends the request to the API Gateway.
- API Gateway performs user authentication using configured protocols (not Django).

- On successful authentication, the API Gateway checks authorization based on pre-defined policies and routes the request to the Node.js backend.
- Node.js retrieves or processes data based on the request specifics (Modbus operations, DB interactions).
- Node.js returns the processed data to the API Gateway.
- API Gateway forwards the data to the React.js frontend.
- Frontend generates and displays relevant graphical reports using the received data.

Benefits of this architecture:

Improved security:

- Separates authentication logic from the Node.js server, reducing its attack surface.

Scalability:

- API Gateway can handle rising user traffic or integrate with different authentication providers.

Flexibility:

- Node.js focuses on Modbus operations, while API Gateway manages all communication and access control.

Reduced complexity:

- Frontend interacts with a single API endpoint (Gateway), simplifying communication.

Additional considerations:

- Implement robust authentication and authorization mechanisms in the API Gateway.
- Secure communication between all components with TLS/SSL encryption.

- Define clear API specifications for the Node.js backend to ensure smooth data exchange.
- Consider implementing caching mechanisms for frequently accessed data.
- Monitor performance and resource usage of each component for optimization.
- Remember, this is a high-level overview, and the specific implementation details will depend on your project's unique requirements and security considerations.

Diagram

Amr system
architecture

